AI-Powered Retail Sales BI Agent – Submission Documentation

1. Executive Summary

Objective: The objective of this project was to build a Business Intelligence Agent that helps non-technical retail managers ask natural-language questions about sales, products, customers, countries, and time trends. The solution uses the Online Retail II dataset, a Supabase PostgreSQL star-schema data warehouse, a Python CLI LLM Agent, a FastAPI MCP-style server, and Apache Superset dashboards.

Key Outcome: The final system allows users to ask business questions, generate safe PostgreSQL SELECT queries, execute them through a protected MCP layer, and view corresponding insights in Superset dashboards. This reduces the need for manual SQL writing and makes retail performance analysis more accessible for business users.

2. Technical Setup and Data Architecture (20%)

Dataset Source: Dataset: Online Retail II
Source: https://www.kaggle.com/datasets/tunguz/online-retail-ii
The dataset contains transactional retail data with invoices, stock codes, product descriptions, quantities, invoice dates, prices, customer IDs, and countries.

Schema Design:
The data warehouse was designed as a star schema. The central fact table stores transactional sales measures, while dimension tables store date, product, and customer attributes.

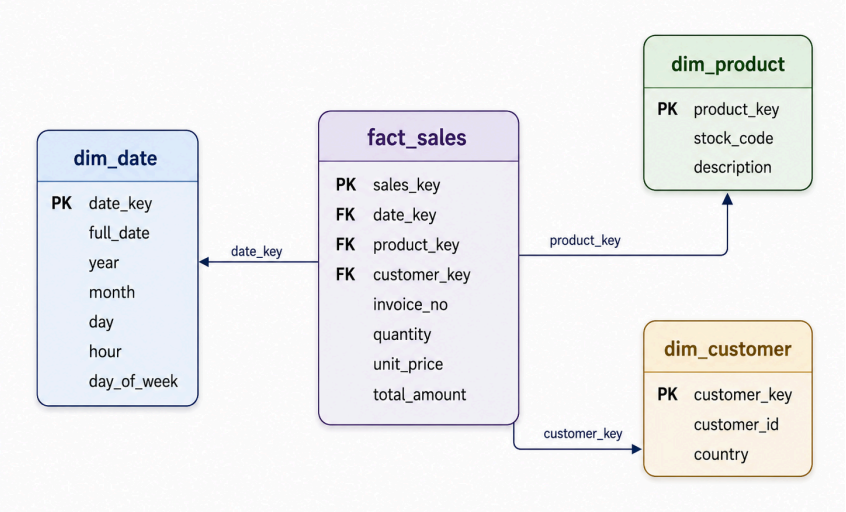

Figure 2.1: Retail Sales Star Schema

Normalization Strategy: The raw Online Retail II file was split into one fact table and three dimension tables. Repeated product, customer, country, and timestamp information was moved into dimension tables. The fact table keeps quantitative measures such as quantity, unit price, and total amount, plus foreign keys pointing to dimensions. This reduces redundancy and supports BI-style aggregation by product, customer, country, and time.

Table 2.1: Fact Table

Table	Primary Key	Foreign Keys	Main Columns
dim_date	date_key	-	full_date, year, month, day, hour, day_of_week
dim_product	product_key	-	stock_code, description
dim_customer	customer_key	-	customer_id, country
fact_sales	sales_key	date_key -> dim_date.date_key product_key -> dim_product.product_key customer_key -> dim_customer.customer_key	invoice_no, quantity, unit_price, total_amount

Data Integrity: Primary keys were defined on each dimension table and on the fact table. Foreign keys in fact_sales enforce relationships with dim_date, dim_product, and dim_customer. This supports referential integrity and ensures that every sales record maps to a valid date, product, and customer record.

ETL and Data Validation: The ETL process was implemented in load_retail_data.ipynb. The notebook reads the raw CSV, handles encoding, removes invalid rows, removes cancelled invoices, calculates total_amount, creates dimension tables, loads data into Supabase, and verifies table row counts. This notebook should be included in the repository under etl/load_retail_data.ipynb because it is direct evidence of the data ingestion pipeline.

Final warehouse row counts: dim_date = 17,282; dim_product = 3,897; dim_customer = 4,346; fact_sales = 397,885.

A validation_checks.py script is also included to compare cleaned raw CSV totals with warehouse totals, including row count and total revenue. This directly supports the evaluation/validity check described in the project instructions.

Environment Setup: Tools used: Python, Jupyter Notebook, pandas, SQLAlchemy, psycopg2, Supabase PostgreSQL, FastAPI, Uvicorn, Gemini API through google-genai, Docker Desktop, Apache Superset, Git/GitHub, and VS Code/Command Prompt.

3. AI Agent Implementation (30%)

- **GitHub Repository Link:** https://github.com/eminaomercevic/AI-Powered-Retail-Sales-BI-Agent
- Reference Boilerplate Repository: https://github.com/dinokeco/bi-agent
- The course BI Agent repository was referenced as the architectural boilerplate. The final implementation adapts the same BI-agent idea to the Online Retail II dataset, Supabase PostgreSQL, a custom Python CLI LLM Agent, and a FastAPI MCP-style server.

Agent Architecture:

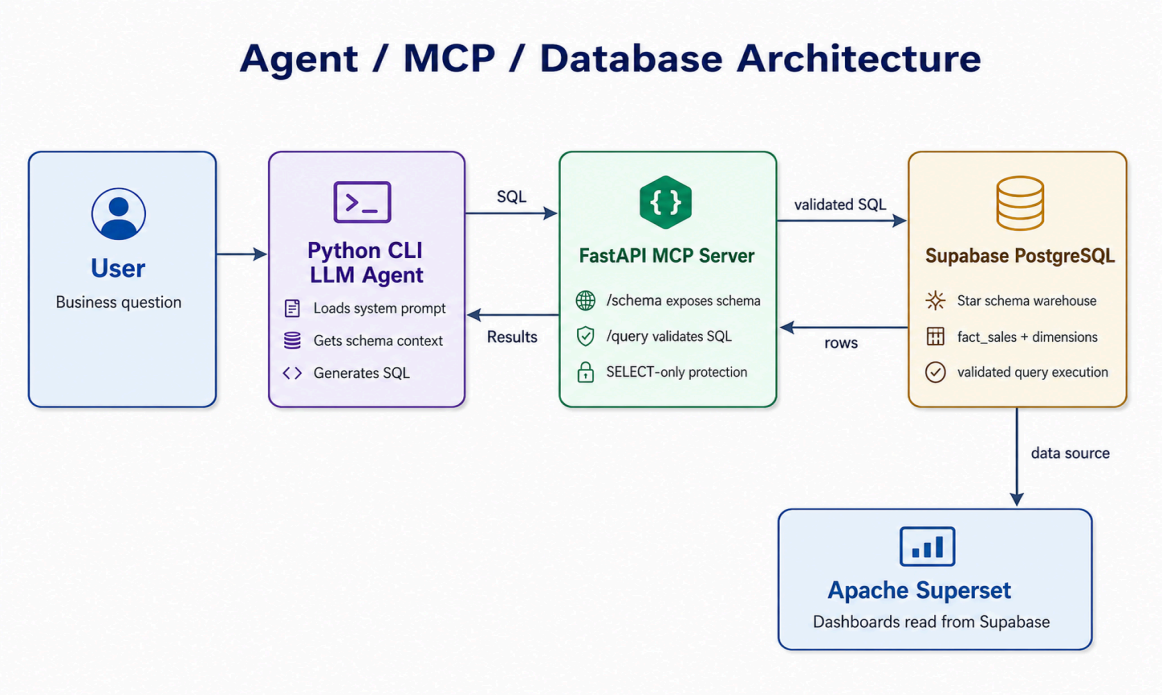

Figure 3.1: Architecture flow diagram

Architecture flow: the user asks a natural-language business question in the Python CLI Agent. The agent loads the system prompt and retrieves live schema context from the MCP server. The LLM generates a PostgreSQL SELECT query. The agent validates the generated SQL and sends it to the MCP /query endpoint. The MCP server validates the SQL again and executes it against Supabase PostgreSQL. Results are returned to the user and can be compared with Superset dashboards.

System Prompting: The core system prompt guides the LLM to generate safe PostgreSQL SELECT queries only, use the available schema context, follow the correct joins, and refuse destructive SQL.

```
You are a Business Intelligence SQL Agent for a retail sales data warehouse.

Your task is to convert natural language business questions into safe PostgreSQL SELECT
queries.
```

```
You are connected to a Supabase PostgreSQL data warehouse through an MCP server. The MCP
server exposes the database schema and executes validated SQL queries.

Rules:

1. Generate only one PostgreSQL SELECT query.

2. Never generate INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, CREATE, GRANT, REVOKE, or
any destructive SQL.

3. Do not include markdown fences.

4. Do not explain the SQL unless asked.

5. Use only the tables and columns provided in the schema context.

6. Always use proper joins from the schema relationships.

7. For revenue, use fact_sales.total_amount.

8. For quantity sold, use fact_sales.quantity.

9. For orders, use COUNT(DISTINCT fact_sales.invoice_no).

10. For customers, use COUNT(DISTINCT dim_customer.customer_id).

11. For product analysis, join fact_sales to dim_product.

12. For customer or country analysis, join fact_sales to dim_customer.

13. For time analysis, join fact_sales to dim_date.

14. If the user asks for top/highest/best, use ORDER BY metric DESC and LIMIT 10 unless the
user specifies another number.

15. If the question cannot be answered from the schema, return exactly:

CLARIFICATION_NEEDED: The question cannot be answered from the available schema.

16. If the user requests destructive SQL, return exactly:

REFUSED: Only safe SELECT queries are allowed.

Available business meaning:

- fact_sales is the central fact table containing transactional sales measures.

- dim_date contains time attributes.

- dim_product contains product information.

- dim_customer contains customer and country information.
```

MCP Configuration: The MCP-style server is implemented with FastAPI in mcp_server.py. It exposes two main endpoints:

- GET /schema - returns tables, columns, and relationships.
- POST /query - accepts a SQL query, validates that it is a SELECT query, blocks dangerous SQL keywords, executes the query against Supabase, and returns rows as JSON.

This design ensures that the LLM Agent does not access the database directly. All database access goes through the MCP layer, which provides schema context and query safety validation.

Error Handling: The system includes two safety layers. First, the agent prompt instructs the LLM to refuse destructive SQL. Second, the MCP server blocks non-SELECT statements even if they are submitted manually.

Failed query example:

```
DROP TABLE fact_sales;
```

MCP server response:

```
400 Bad Request
 {"detail": "Only SELECT queries are allowed."}
```

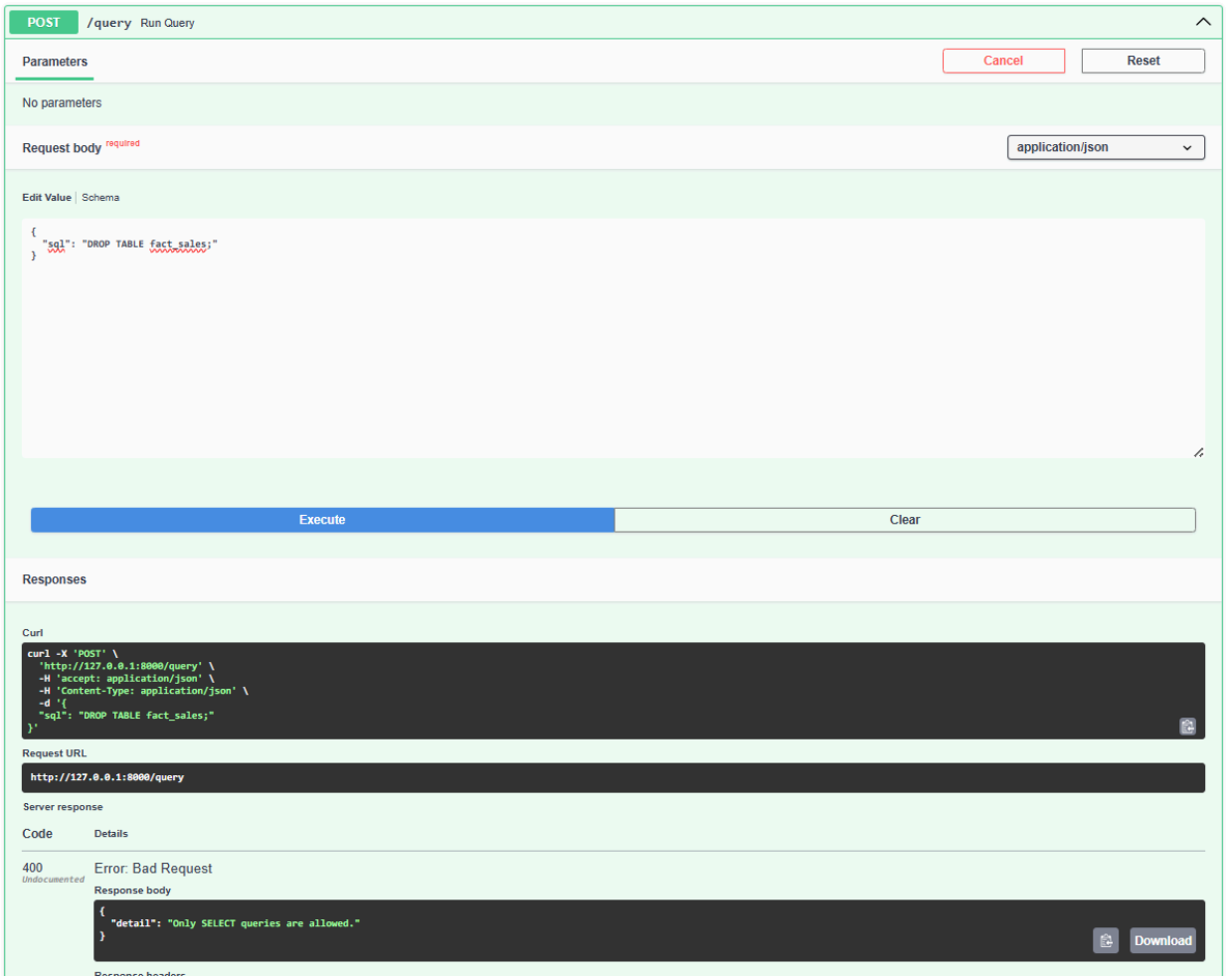

Figure 3.2: MCP server response

4. BI Dashboarding (Superset) (25%)

Connectivity: Apache Superset was run locally using Docker and connected to Supabase PostgreSQL using a SQLAlchemy PostgreSQL connection string. A virtual dataset named retail_sales_full was created by joining fact_sales with dim_date, dim_product, and dim_customer. This dataset was used for all Superset charts.

Metrics Definitions:

Table 4.1: Metrics Definitions

Metric	Definition
Total Revenue	SUM(total_amount)
Total Orders	COUNT(DISTINCT invoice_no)
Total Customers	COUNT(DISTINCT customer_id)
Average Order Value	SUM(total_amount) / COUNT(DISTINCT invoice_no)
Product Revenue	SUM(total_amount) grouped by product_name
Quantity Sold	SUM(quantity)
Country Revenue	SUM(total_amount) grouped by country
Monthly Revenue	SUM(total_amount) grouped by full_date/month

Visualizations:

1. **Executive Overview**

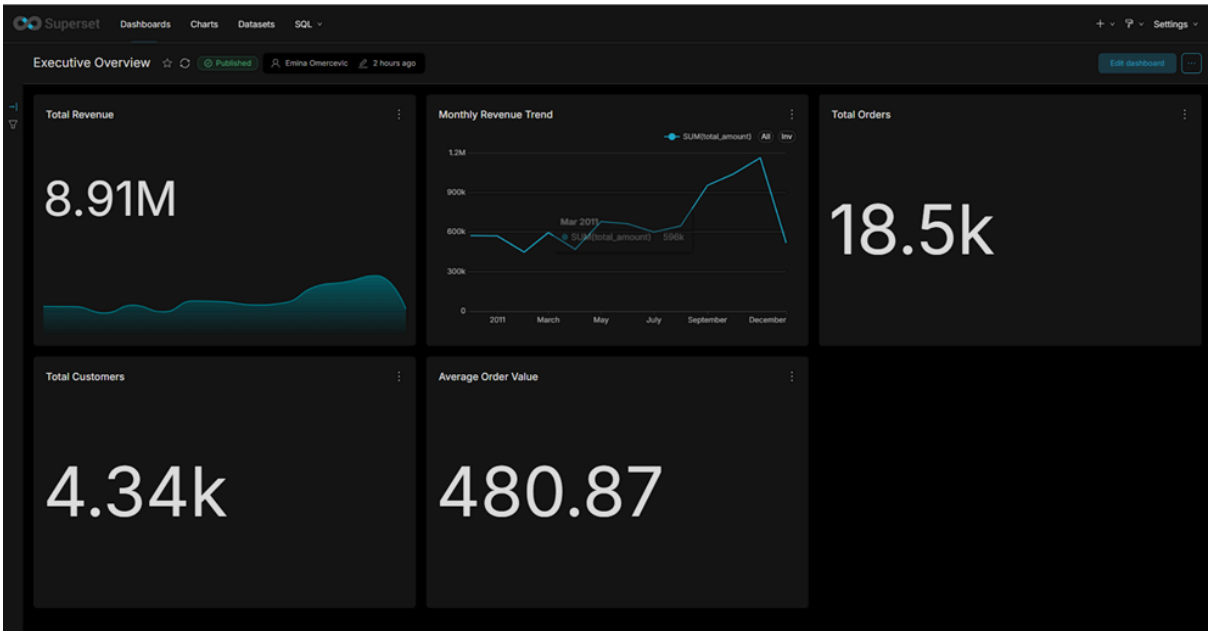

Figure 4.1: Executive Overview Dashboard

2. Operational Deep-Dive / Product Performance

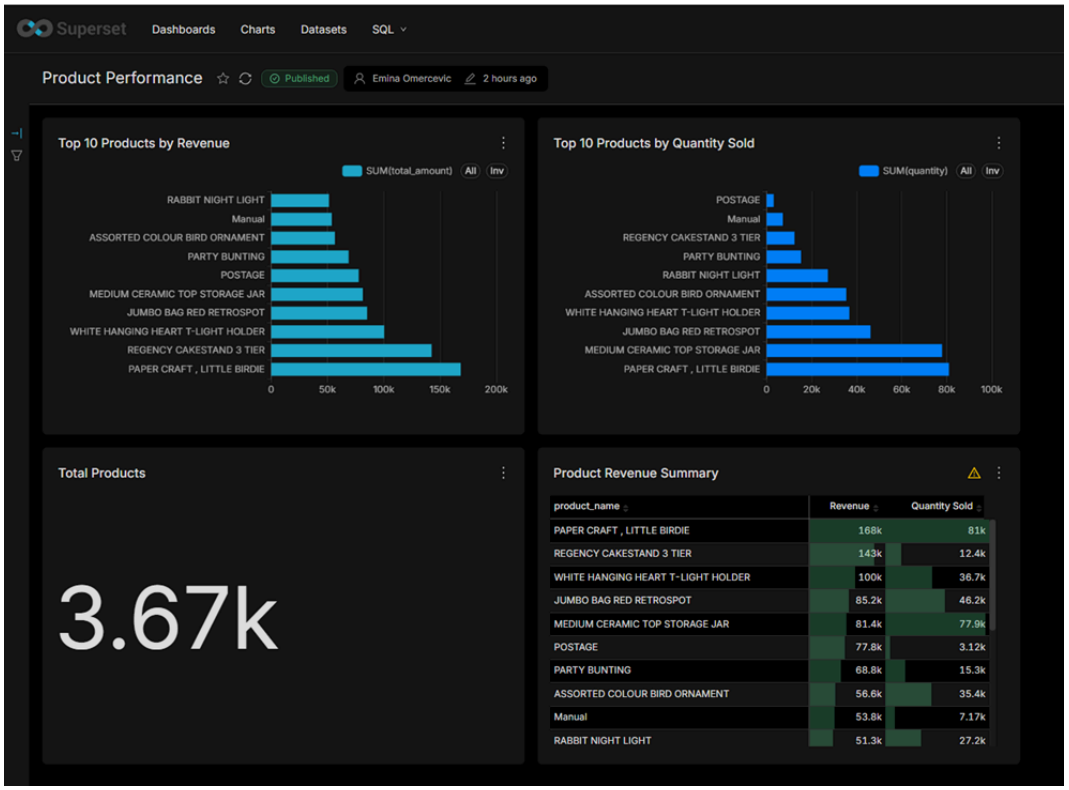

Figure 4.2: Operational Deep-Dive / Product Performance Dashboard

3. Trend/Anomaly Monitor / Customer & Country Analysis

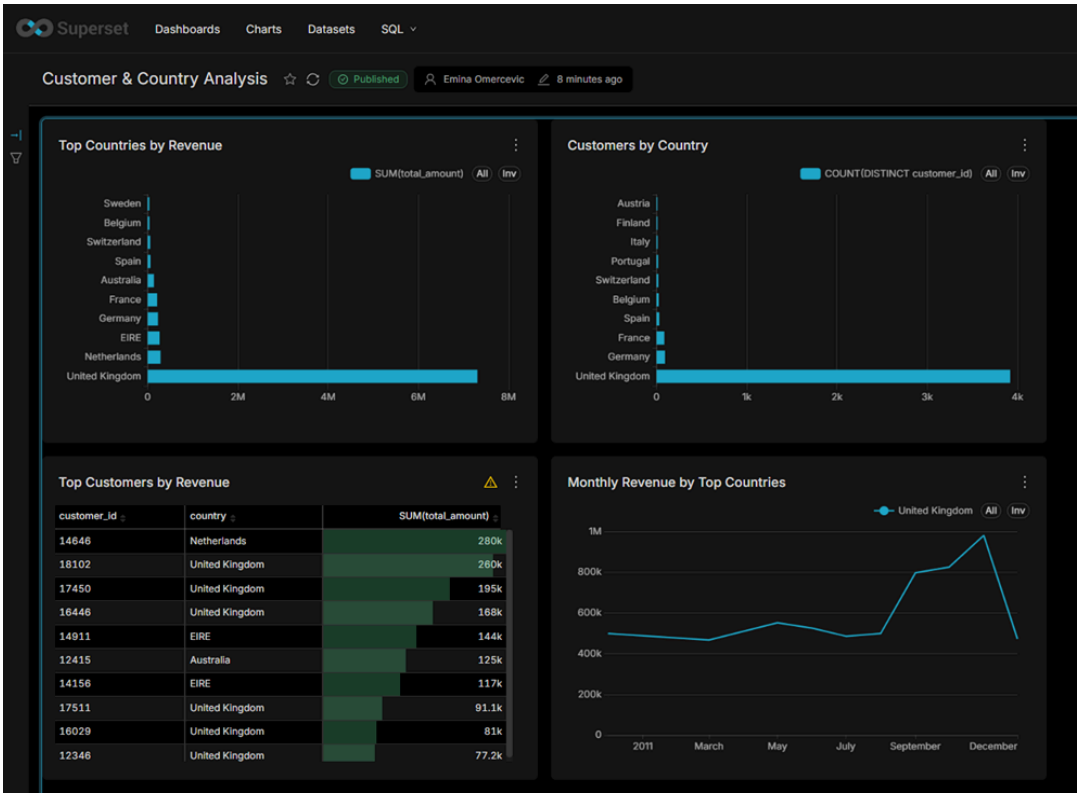

Figure 4.3: Trend/Anomaly Monitor / Customer & Country Analysis Dashboard

5. Final Presentation and Demo (25%)

Business Value Analysis: The BI Agent supports decision-making by allowing managers to ask natural-language questions instead of manually writing SQL. The target users are non-technical retail managers, business analysts, sales managers, and operations teams who need quick answers about sales performance, product performance, customers, and countries.

Three Data Insights

- United Kingdom strongly dominates both revenue and customer count, which shows a heavy geographic concentration risk and opportunity.
- PAPER CRAFT, LITTLE BIRDIE is the top product by revenue, while quantity-based ranking shows that high-volume and high-revenue products are not always the same.
- Monthly revenue rises sharply toward the later months of 2011, especially around November, suggesting a seasonal retail peak that should guide inventory and marketing planning.

Architectural Overview:

Supabase was chosen because it provides a managed PostgreSQL database with relational integrity and support for primary/foreign keys. The MCP server was used as a secure database access layer between the LLM Agent and the database. Superset was chosen because it provides open-source BI dashboarding and can connect directly to PostgreSQL.

Efficiency Gains

Without the agent, users would need to know SQL syntax, table relationships, and metric definitions. With the agent, users can ask business questions directly and receive SQL-backed answers. This improves speed, reduces query-writing errors, and helps standardize metrics through the MCP and Superset layers.

Data Flow

Kaggle Online Retail II CSV -> Python/Jupyter ETL -> Supabase PostgreSQL star schema -> MCP server schema/query endpoints -> Python CLI LLM Agent -> Apache Superset dashboards.

- **Video Demo Link:** AI-Powered Retail Sales BI Agent - VIDEO.mp4
- **Presentation Link:** AI-Powered Retail Sales BI Agent - PRESENTATION

Reflection on Technical Challenges: One major challenge was setting up Superset locally and connecting it to Supabase. Docker initially had network/DNS issues and the Superset container lacked the PostgreSQL driver. This was solved by using a prebuilt Superset Docker image, adding the required database driver, and testing the Supabase connection through SQL Lab before building dashboards.

Another challenge was implementing the agent correctly. The first version used predefined SQL patterns, which was not enough for a true LLM-guided agent. The improved version loads system_prompt.txt, retrieves schema context from the MCP server, sends the question and schema to the LLM, validates the generated SQL, and then executes it through the MCP server.

LINKS:

- **GitHub Repository Link:** https://github.com/eminaomercevic/AI-Powered-Retail-Sales-BI-Agent
- **Video Demo Link:** AI-Powered Retail Sales BI Agent - VIDEO.mp4
- **Presentation Link:** AI-Powered Retail Sales BI Agent - PRESENTATION